

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Assessment Tool Weightage: 20%

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I **Y. Sai Mohith(192372210)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Y. Sai Mohith

Assessment Tasks

Case Study

1. Case 1: A food delivery company wants to deploy a cloud-native backend with web APIs, browser clients, and shared cloud storage for receipts and delivery images. Propose the architecture and explain the role of each component.

Answer

A suitable architecture for the food delivery company should use a cloud-native, layered design in which browser clients communicate with web APIs and the backend uses managed cloud services for data storage and shared files. The main components can be organized into the client layer, API layer, application layer, data layer, object storage layer, and monitoring and security layer. This design allows the company to handle a large number of customers, restaurants, delivery partners, and orders without depending on a single server.

The browser client is the front-end through which customers can register, log in, search restaurants, view menus, place orders, make payments, and track deliveries. Delivery partners and restaurant staff can also use browser-based interfaces for accepting orders and updating delivery status. The clients should communicate with the backend through secure HTTPS web APIs rather than directly accessing databases or storage. APIs provide a clear interface between the user interface and cloud services and make it easier to develop, test, maintain, and scale the application.

The API gateway or load-balanced API endpoint can act as the main entry point for requests. It can route requests to several backend application instances. For example, one request may be sent to the authentication service, another to the restaurant service, and another to the order or delivery service. Authentication and authorization should be applied at the API layer so that only permitted users can access protected operations. Rate limiting can also be used to prevent excessive requests from affecting the service.

Behind the API layer, the company can deploy cloud application services or virtualized containers for business logic. Separate services can manage users, restaurants, menus, orders, payments, delivery tracking, notifications, and receipts. Running multiple instances of these services improves availability. Auto-scaling can increase the number of instances when demand rises during lunch hours, weekends, or special offers and reduce them when traffic falls. Load balancing distributes incoming requests among healthy instances, preventing one server from becoming overloaded.

For structured application information such as customers, restaurants, orders, payment status, and delivery records, a managed cloud database is appropriate. The database should provide backups, replication, access control, and monitoring. Database indexing and suitable queries can improve performance. Frequently requested information, such as restaurant menus or popular items, can also be cached to reduce repeated database access. Transaction processing should be designed carefully so that an order is not duplicated or lost when several services operate at the same time.

Shared cloud storage should be used for receipts, delivery images, invoices, and other files. Object storage is suitable because it can store large numbers of files and can scale without requiring the company to manage physical disks. Each object can be associated with metadata such as order ID, customer ID, file type, and upload time. Access should be controlled using identity and access management policies. Sensitive files should be encrypted, and temporary access links can be used when a customer or delivery partner needs to download a particular receipt or image.

The architecture should also include monitoring, logging, and alerting. Metrics such as API response time, error rate, CPU usage, memory usage, database connections, storage usage, and request volume should be monitored. Logs can help developers identify failed payments, API errors, authentication problems, or image-upload failures. Alerts should be configured for abnormal conditions so that administrators can take corrective action quickly.

Security is important because the system handles personal information, addresses, order details, and payment-related information. HTTPS should protect communication between clients and APIs. Strong authentication, role-based access control, encryption, secure secrets management, and regular security testing should be implemented. Restaurant users, delivery partners, customers, and administrators should receive only the permissions required for their roles.

Overall, the proposed architecture separates the client, API, application, database, and storage responsibilities. It supports scalability through auto-scaling and load balancing, reliable file storage through cloud object storage, and secure communication through web APIs and identity controls. The architecture is practical because each component can be managed and scaled according to its workload. It would allow the food delivery company to start with a smaller deployment and increase capacity as the number of customers and orders grows.

2. Case 2: A media startup must decide between SAN-backed storage, object storage, and serverless processing for video upload workflows. Analyze the case and recommend a deployment model.

Answer

For the media startup, the three options—SAN-backed storage, object storage, and serverless processing—solve different parts of the video upload workflow. A good deployment model should therefore select the technology according to the characteristics of video files, the expected workload, scalability requirements, and the amount of infrastructure management the startup can support. For a cloud-native media platform, object storage combined with serverless processing is generally the most suitable model, while SAN-backed storage can be used only when the application has a specific requirement for low-latency block storage.

SAN-backed storage provides block-level storage that can be attached to servers or virtual machines. It can offer predictable performance and is useful for applications that require traditional file systems or frequent random read and write operations. However, video platforms usually generate very large files and may store millions of objects. Scaling a SAN environment for such workloads can become expensive and can require more administration. It may also create capacity-planning problems because storage has to be provisioned in advance.

Object storage is better suited to video upload workflows because each video can be stored as an object with metadata. It is designed for large-scale unstructured data such as videos, images, documents, and backups. The storage system can grow as the number and size of videos increase, reducing the need for manual disk management. A video can be assigned an object key based on information such as user ID, video ID, and upload date. Metadata can record the file type, duration, upload status, and processing state.

The upload process can begin when a user selects a video through a browser or application. Instead of sending the complete file through the main application server, the backend can generate a secure temporary upload permission. The client can then upload the video directly to cloud object storage. This reduces load on the application server and allows many users to upload large files at the same time. Multipart or resumable uploads can be used for very large videos so that an interrupted upload does not always have to restart from the beginning.

After a video is successfully uploaded, an event can trigger a serverless processing function. The function can validate the uploaded object, create a processing job, update metadata, and start transcoding or thumbnail generation. Serverless processing is useful because the startup does not need to keep processing servers running continuously. Resources can be invoked when uploads arrive and released after the task is completed. This provides automatic scaling when the number of uploads increases.

For large video transcoding tasks, a managed media-processing service or container-based processing system can work together with serverless functions. The serverless function can coordinate the workflow while the actual transcoding service converts the original video into multiple resolutions and formats. For example, a single uploaded video may be converted into lower, medium, and high-quality versions so that users with different network speeds can stream suitable content. Generated files can again be stored in object storage.

A content delivery network can be placed in front of the processed video files so that frequently accessed content is cached closer to viewers. This reduces latency and decreases repeated requests to the origin storage. Storage lifecycle policies can also move older or rarely accessed videos to lower-cost storage classes. Deletion policies can remove temporary files after a defined period. These features reduce the total cost of long-term storage.

Security should be implemented throughout the workflow. Access to storage should be controlled using identity and access management policies. Private buckets or containers should be used for original uploads, and users should receive only authorized access. Data should be encrypted during transfer and at rest. Logging should record upload, processing, access, and deletion activities. Backup and versioning policies can protect important media from accidental deletion or corruption.

Therefore, the recommended deployment model is object storage as the primary repository, serverless functions for event-driven workflow control, managed media processing for heavy transcoding, and a CDN for content delivery. SAN-backed storage should be considered only for specialized processing components that require block-level storage. This model is scalable, flexible, and cost-efficient for a startup because storage expands with demand and processing resources are used when required. It also reduces infrastructure management and allows the company to concentrate on media features rather than maintaining physical storage systems.

3. Case 3: A smart city platform needs low-latency decision making near sensors while still using the cloud for analytics. Analyze the use of edge computing and fog computing in this environment.

Answer

The smart city platform has two different requirements: very fast decisions must be made close to sensors, while large-scale analysis can be performed in the cloud. A combination of edge computing and fog computing is therefore appropriate. Edge computing places processing capability directly near or on devices such as cameras, traffic sensors, environmental sensors, and smart meters. Fog computing provides an intermediate processing layer between edge devices and centralized cloud services. Together, they reduce unnecessary data transfer and improve the response time of time-sensitive applications.

In a traditional cloud-only design, sensors continuously send raw data to a central cloud region. The cloud processes the information and sends a decision back to the device. This approach can work for applications that tolerate delay, but it may not be suitable for traffic control, emergency detection, industrial monitoring, or public safety. Network delay, congestion, and temporary connectivity problems can prevent a decision from being made quickly. Sending every raw sensor reading to the cloud can also consume significant bandwidth.

With edge computing, immediate processing can occur at or very close to the source of the data. For example, a traffic camera can use an edge device to identify vehicle density and detect an accident without sending every video frame to the cloud. A temperature sensor can detect an abnormal value locally and immediately trigger an alert. Only important events, summaries, or selected data need to be sent to higher layers. This reduces latency and network traffic.

Fog computing adds a regional or local processing layer between edge devices and the cloud. A group of traffic sensors in one area could send information to a nearby fog node. The fog node can combine

readings from multiple sensors, perform more complex analysis, and coordinate local decisions. For example, several intersections can share traffic information through a fog layer and adjust traffic signals according to congestion. This provides more processing power than individual edge devices while still being closer to the sensors than a centralized cloud.

The cloud remains important for long-term analytics and centralized management. Aggregated data from edge and fog layers can be sent to cloud storage for historical analysis. Cloud-based analytics can identify traffic patterns, pollution trends, energy consumption, equipment failures, and other city-wide conditions. Machine learning models can be trained using large datasets in the cloud and then suitable models can be deployed back to edge devices or fog nodes for local decision making.

A practical architecture can therefore contain four layers. The first layer is the sensor and device layer, which collects data. The second is the edge layer, which performs immediate filtering and event detection. The third is the fog layer, which aggregates information from several edge locations and performs regional processing. The fourth is the cloud layer, which provides centralized storage, large-scale analytics, dashboards, reporting, model training, and system management.

Security must be considered at every layer. Devices should use secure identities and authenticated communication. Data should be encrypted while moving between sensors, fog nodes, and cloud services. Access control should ensure that only authorized services can read or modify data. Logging and monitoring should detect unusual behavior. Devices and edge software should also be updated regularly to reduce security vulnerabilities.

Edge and fog computing also improve resilience. If the connection to the central cloud is temporarily unavailable, local edge or fog systems can continue performing critical decisions. For example, a traffic signal can continue using locally available information rather than waiting for a cloud response. When connectivity is restored, stored or summarized information can be synchronized with the cloud. This creates a more fault-tolerant system.

However, the approach also introduces challenges. Edge devices have limited computing and storage resources, and managing many distributed devices can be difficult. Fog nodes require physical security, monitoring, software updates, and reliable connectivity. Data consistency between edge, fog, and cloud layers must also be managed carefully. The organization must balance local processing with centralized control.

Overall, edge computing should be used for immediate, low-latency decisions, fog computing for regional aggregation and processing, and cloud computing for large-scale analytics and centralized management. This combined architecture is well suited to smart city applications because it reduces latency, saves bandwidth, supports operation during connectivity failures, and still provides the cloud capacity required for long-term analysis and intelligent city planning.